

第 11 章 PCI/PCIe 总线的序

在一个处理器系统的实现中，开发者需要注意两个问题，一个是 Cache 的一致性 (Cache Coherency)，另一个是数据的完整性 (Data Consistency)。深入理解这两部分内容的过程贯穿处理器体系结构学习的始终。在一个处理器系统的实现过程中，由这两个问题引发的系统错误是最难定位的。因为这两种系统错误的表现形式很难与“Cache 共享一致性”与“序”联系在一起，即便最后定位是这两种原因引发的系统错误，也不容易复现这些系统错误。

本章主要讲述在 PCI/PCIe 总线中，数据传送的“序”与可能出现的死锁。在总线体系结构的设计中需要重点考虑序与死锁的问题，序和死锁一直是系统架构设计的重点所在，也是逻辑性要求较强、最容易出错、系统程序员容易忽略的内容。

所谓“序”是指数据传送的顺序，是保证数据完整性的基础。而死锁是指两个以上的设备在访问临界资源时，相互等待对方释放这些资源，而无法访问这些资源的情况。合理地安排访问“序”是解决死锁的一个有效方法。

在 PCI/PCIe 总线中，序与生产/消费者模型密切相关。生产/消费者模型是一种并发协作模型，PCI/PCIe 设备使用该模型进行数据传递。在 PCI/PCIe 总线中，访问“序”的安排必须保证生产/消费者模型的正确运转，这也意味着在 PCI/PCIe 总线中，数据的传送规则需要与生产/消费者模型一致。

合理安排数据访问的“序”对于一个系统设计是至关重要的，同时也是一个系统设计的基础。对于一个专用系统，生产消费者模型使用的数据缓冲、Flag 和 Status 位在系统中的位置相对固定，以此为基础设计合理的数据访问“序”也许并不困难。

但是对于一个通用处理器系统，数据缓冲、Flag 和 Status 位的在系统中位置并不固定，此时合理安排数据访问“序”需要异常缜密的思维。而 PCI/PCIe 总线针对的就是这样一个通用处理器系统。

本章将在第 11.3 节介绍 PCI 总线的序，并在第 11.4 节详细介绍 PCIe 总线的序。其中 PCIe 总线的序基于 PCI 总线的序，并适当简化了 PCI 总线的强序模型，补充了 PCIe 总线的“Relaxed Ordering”和 IDO (ID-Base Ordering) 模型。

11.1 生产/消费者模型

除了在 PCI/PCIe 总线中，互连网络中的数据传递以及多进程间的数据通信也经常使用生产/消费者模型。生产/消费者模型的正确运行需要一些必要条件，该模型由以下几个基本单元组成。

- 共享数据缓冲。由生产者写入，消费者读取的数据区域。
- 生产者。数据的提供方，生产者需要产生数据，并在消费者将数据读出后，再将数据写入缓冲中。

- 消费者。数据的使用方，消费者将消费数据，并在生产者将数据写入共享缓冲后，再获取数据。
- Flag 位。生产者通过对该位写 1 通知消费者，已经将数据写入缓冲中。消费者通过该位判断数据缓冲是否有效，为 1 表示在数据缓冲中的数据已经被生产者写入；为 0 表示没有被写入。该位由生产者写 1，由消费者清零。
- Status 位。消费者通过对该位写 1 通知生产者，已经将数据从缓冲读出；生产者通过该位判断数据缓冲是否有效，为 1 表示在数据缓冲中的数据已经被消费者读出；为 0 没有读出。该位由消费者写 1，由生产者清零。

生产/消费者模型需要使用两个状态位完成数据的交换。因为生产者和消费者在使用数据缓冲时，需要多种数据状态。这些数据状态如表 11-1 所示。

表 11-1 Flag 和 Status 状态位

Flag	Status	描 述
0	0	空闲状态，此时数据缓冲为空
1	0	生产者写入数据，但是消费者没有使用该数据
0	1	消费者使用完毕该数据
1	1	正常情况不会出现该状态

11.1.1 生产/消费者的工作原理

在生产/消费者模型初始化时，Flag 和 Status 位都为 0，之后生产者和消费者通过操纵和检测 Flag 和 Status 位，进行数据传递。其中生产者负责将数据填入到数据缓冲中，而消费者负责将数据从缓冲中取出，其实现过程如表 11-2 所示。

表 11-2 生产/消费者的工作流程

	生产者工作流程	消费者工作流程
1	将数据写入缓冲中	查询 Flag 位，直到该位为 1
2	置 Flag 位为 1	置 Flag 位为 0
3	查询 Status 位，直到该位为 1	将数据从缓冲中读出
4	置 Status 位为 0。如果有数据发送，转（1）	置 Status 位为 1，转（1）

由以上描述，可以发现生产者仅轮询 Status 位，在条件允许时将该位清零，而直接对 Flag 位写 1；消费者仅轮询 Flag 位，并在条件允许时将该位清零，而直接对 Status 位写 1。

设计者在使用生产/消费者模型时，需要注意两个竞争条件。

1) 由于生产者对“Flag 位置 1”和“数据写入数据缓冲”可能没有采用相同的路径，因此这两个动作不一定同步进行。当生产者将 Flag 位置 1 时，可能数据没有完全写入到缓冲中。但是在消费者查询 Flag 位已经为 1，并将该位清零之后，数据一定要写入缓冲中，否则消费者将得到无效数据。

2) 消费者对“数据从缓冲读出”和“Status 位置 1”也可能没有采用相同的路径，这两个动作也不一定同步。因此消费者将 Status 位置 1 时，可能数据没有完全从缓冲中读出。

但是在生产者查询 Status 位已经为 1，并将该位清零之后，数据一定要从缓冲读出，否则生产者将会清除缓冲中的有效数据。

由此可见，生产/消费者模型的正确运行是有条件的。为此在一个实际的系统应用中，必须合理安排数据访问的“序”，使得该模型能够正常运转。而且这个“序”需要考虑在一个系统中存在多个生产者、多个消费者和多个数据缓冲的情况。

11.1.2 生产/消费者模型在 PCI/PCIe 总线中的实现

生产/消费者模型在 PCI/PCIe 总线中得到了充分的应用。两个 PCI 主设备之间进行数据传送，或者 PCI 主设备进行 DMA 操作时需要使用该模型。PCI 总线规定，PCI 设备必须按照生产/消费者模型提供的规则访问存储器或者 I/O 资源。采用该模型对于 PCI 设备访问存储器或者 I/O 资源时避免死锁和保证数据完整性至关重要。PCI/PCIe 总线规定了数据访问的顺序，本章在第 11.3 和第 11.4 节将讲述这些内容。

下文以两个 PCI 主设备进行数据交换说明生产/消费者模型的实现机制，假设这两个 PCI 主设备在进行数据交换时使用一个数据缓冲和 Flag 和 Status 这两个状态位。

在 PCI 总线中，数据传送只要遵循合理的顺序，无论生产者、消费者在哪一级 PCI 总线上，数据缓冲和 Flag、Status 状态位在处理器系统的什么位置，都能准确无误地发送和接收数据，保证生产/消费者模型的正确运转。

下面以图 11-1 为例，进一步说明在 PCI 总线中，生产/消费者模型的运转过程。其中消费者与数据缓冲在同一条 PCI 总线上，而 Flag 位、Status 位和生产者在另外一条 PCI 总线上。这两条 PCI 总线之间通过一个 PCI 桥进行连接。在这种处理器系统中，生产/消费者模型的详细运转过程见表 11-3。

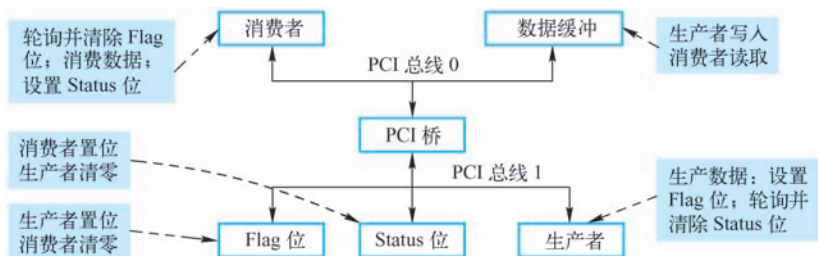


图 11-1 生产/消费者模型的实例

PCI 桥的存在为生产/消费者模型在 PCI 体系结构中的实现带来了新的挑战，PCI 桥可以缓存一些报文，因此来自 PCI 设备的报文并不能立即到达目的地。其中 Flag 和 Status 位的初始值为 0。

表 11-3 基于 PCI 总线的生产/消费者模型的详细描述

步 骤	操 作	描 述
1	生产者通过 PCI 桥，使用 Posted 方式将数据写入数据缓冲中	生产者和数据缓冲不在同一条总线上，因此生产者首先使用 Posted 写周期将数据写入 PCI 桥，并不会立刻到达数据缓冲
2	生产者将 Flag 位置 1	Flag 位和生产者在同一条总线上，Flag 位会被立即置位

(续)

步 骤	操 作	描 述
3	生产者轮询 Status 状态位, 判断数据缓冲中的数据是否被消费者处理完毕	Status 位和生产者在同一条总线上, 生产者可以立即获得 Status 位的信息
4	消费者轮询 Flag 状态位	Flag 位和消费者不在同一条总线上, 因此消费者首先将读请求发向 PCI 桥, PCI 桥将使用 Delayed 读总线请求周期获得数据, 同时可能使用“Retry”周期结束消费者的读请求。为保证生产/消费者模型的正确运行, PCI 桥必须进行以下操作读取 Flag 位。 1. 为了保证读请求的正确性, PCI 桥需要首先将与读请求相同方向的 Posted 写请求发送出去, 即发送到 PCI 总线 1 上。 2. PCI 桥读取 Flag 位。 3. PCI 桥将所有 Posted 写请求发送到 PCI 总线 0 上。 4. PCI 桥将 Flag 信息发送给消费者
5	如果消费者发现 Flag 位被生产者置为 1, 则将 Flag 位清零	这个清零操作首先也是发向 PCI 桥, 并不会立刻到达 Flag 位
6	消费者从数据缓冲中获得数据	消费者与数据缓冲在同一条总线上, 可以直接获得数据
7	消费者处理完毕这些数据后, 将 Status 位置 1	消费者和 Status 位不在同一条总线上, 因此生产者首先使用 Posted 写周期将数据写入 PCI 桥, 并不会立刻更新 Status 位
8	消费者继续轮询 Flag 位, 确定数据缓冲中是否有新的数据需要处理	消费者再一次读取 Flag 位时, 仍然需要跨越 PCI 桥, 此时强制将 PCI 桥中的 Posted 写总线事务刷新出去
9	生产者读取 Status 位, 并依此判断消费者是否处理完毕数据缓冲中的数据	生产者和 Status 位在同一条总线上, 因此可以立即获得 Status 位的信息
10	如果 Status 位为 1, 则生产者将 Status 位清零	生产者和 Status 位在同一条总线上, Status 位可以立即被更新
11	如果生产者继续提供数据, 将重新启动整个生产/消费者模型	

由以上过程可以发现, 由于 PCI 桥的存在数据并不能立即到达目的地, 因此有可能造成总线死锁和数据不完整等一系列问题, 最终导致生产/消费者模型不能在 PCI 总线上正确运行。为了解决这个问题, PCI 总线规定了一系列与数据传送“序”有关的规则。

如果 PCI 设备不满足这个“序”的要求, 就有可能出现数据完整性的问题, 从而导致数据传送失败。如果读者有机会设计基于 PCI/PCIe 总线的设备, 需要认真考虑这个序的问题。值得注意的是, PCIe 总线的序与 PCI 总线的序略有区别, 在第 11.4 节将详细讨论 PCIe 总线的序。

在 PCI/PCIe 总线中, 使用 Posted 总线请求比 Non-Posted 总线请求传送数据的延时更短。因此在一个具体的实现中, 如果 Status 位距离生产者的路径较近时, 轮询该位的代价较低; 同理 Flag 位距离消费者的路径较近时, 轮询该位的代价也较低。

值得注意的是, 在 PCI 体系结构中, 无论生产者、消费者、Flag 和 Status 位在处理器系统的哪级 PCI 总线中, 生产/消费者模型都可以正常运行, 为此 PCI 总线详细规定了数据传送的顺序, PCI 总线的“序”对于理解 PCI 体系结构至关重要, 读者需要重视这部分内容。

本书建议读者改变图 11-1 中生产者、消费者、Flag 位和 Status 位的位置，按照第 11.3 节提供的规则，验证生产/消费者协议在 PCI 总线中的正确性。

11.2 PCI 总线的死锁

Tom Shanley 与 Don Anderson 对 PCI 总线的死锁进行了详细说明，本节首先通过两个实例说明在 PCI 总线中存在的死锁问题。其中第 1 个实例是因为缓冲管理不慎而导致的死锁，而第 2 个实例是因为数据传送的顺序而导致死锁。本节更侧重介绍因为数据的传送顺序而导致的死锁。

11.2.1 缓冲管理引发的死锁

如图 11-2 所示，假设在一条 PCI 总线上有两个非桥设备 A 和 B。其中 A 和 B 之间相互进行存储器写操作，这个存储器写操作需要使用设备 A 和 B 内部的缓冲区。而且设备 A/B 的发送部件和接收部件共享同一个数据缓冲。在图 11-2 的左图中 PCI 设备 A/B 的发送和接收缓冲共用了一个数据缓冲，而在右图中将这两者分离。



图 11-2 PCI 总线中的死锁实例 1

设备 A 和 B 进行存储器写操作的流程如下。

- 1) 设备 A 和 B 同时申请 PCI 总线资源，同时 A 和 B 已经将即将发送的数据写入各自的缓冲区中，准备进行数据传递。
- 2) 设备 A 通过总线仲裁优先使用 PCI 总线，同时将缓冲区中的数据发向设备 B。
- 3) 设备 B 因为在缓冲区中尚有数据需要传递，因此使用重试周期拒绝了设备 A 的数据请求，并获得 PCI 总线的使用权，将缓冲区中的数据发向设备 A。
- 4) 设备 A 也因为在缓冲区中尚有数据需要传递，而使用重试周期拒绝了设备 B 的数据请求。
- 5) 在这种情况下，设备 A 和设备 B 都因为对方的缓冲正在被使用，而无法完成存储器写，从而产生了死锁。

从这个例子中可以发现，如果 PCI 设备对缓冲区的管理不慎，极易造成死锁。我们可以采用一种简单的方法解决这类死锁问题，只要设备 A、B 将接收和发送使用的缓冲分离，这样可以保证在步骤 3) 中，设备 B 可以接收来自设备 A 的数据，从而避免了死锁。

11.2.2 数据传送序引发的死锁

本节首先分析一个在 PCI 总线中死锁的实例。假设在 PCI 桥 A 中存放了一个 Posted 写请求，该请求正在准备发向 Secondary PCI 总线。而在 PCI 桥 A 还没有获得 Secondary PCI 总线的使用权时，Secondary 总线上的 PCI 设备 B，需要使用 Delayed 总线事务通过该 PCI 桥从主存储器中读取数据。如图 11-3 所示。

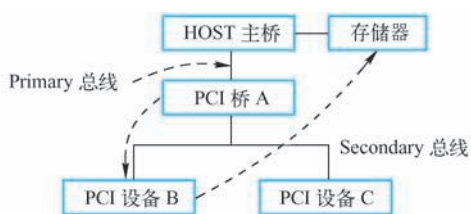


图 11-3 PCI 总线中死锁实例 2

PCI 桥 A 与 PCI 设备 B 的工作流程如下所示。

1) 首先 PCI 设备 B 发出的存储器读请求已经从存储器中获得数据，HOST 主桥将这个存储器读请求转换为存储器读完成。当这个存储器读完成穿越 PCI 桥时，要求刷新 PCI 桥中的 Posted 写请求（从 Primary 总线到 Secondary 总线的 Posted 写请求）。因此 PCI 桥首先需要重试来自 PCI 设备的读完成，然后将存储在 PCI 桥中的这个 Posted 写请求刷新出去，详细原因见第 11.3.1 节。

2) 如果这个 Posted 写请求的目的设备恰好是需要从存储器读取数据的 PCI 设备，此时将可能发生死锁。假设在 PCI 桥 A 暂存的 Posted 写的目的地恰好为 PCI 设备 B。

3) PCI 设备 B 发现有发向自己的 Posted 写请求时，并不接收这个请求，而是使用重试周期拒绝这个写请求，因为这个 PCI 设备希望从存储器读取完数据后，才能接收这个写请求。

4) 此时在 PCI 桥 A 中暂存的 Posted 写无法完成，同时 PCI 设备 B 的读完成请求也无法穿越 PCI 桥 A，此时将产生死锁。

产生这个死锁的原因是 PCI 设备 B 需要完成存储器读之后才能接收 Posted 存储器写，如果 PCI 设备 B 可以先接收 Posted 存储器写，之后再继续进行存储器读完成请求，这个死锁就可以避免。产生该死锁的原因与 PCI 总线的序相关，合理地安排这些总线事务的访问序将可以避免这类死锁，下文将详细介绍 PCI 总线的序。

在 PCI 总线中，如果没有合适地处理序的问题，将产生多种类型的死锁。本节所讲述的仅是其中一个较为简单的实例。下文将继续介绍有关 PCI 总线序引发的死锁问题。PCI 总线通过安排访问顺序，可以合理地解决这类死锁问题。

11.3 PCI 总线的序

PCI 总线为满足生产/消费者模型的正确运转，设置了许多与“序”相关的规则。只要 PCI 设备满足这些序，那么无论数据缓冲、Flag 和 Status 位在 PCI 总线的什么位置，都可以保证生产/消费者模型的正常运转。

11.3.1 PCI 总线序的通用规则

PCI 总线在进行数据传递时规定了一些规则。

1) PCI 总线仅支持 Posted 存储器写总线事务，而配置和 I/O 写总线事务只能使用“Delayed”总线写事务实现。

2) Posted 存储器写总线事务需要按序完成。PCI 桥必须按照“先进先出”的原则，处

理 Posted 存储器写总线事务。在图 11-1 所示的实例中, 如果 Flag 位在 PCI 总线 0 上时, 生产者需要通过 PCI 桥传递数据, 并将 Flag 位置 1, 这两个操作都需要使用 Posted 存储器写总线事务。如果 PCI 不遵循“先进先出”的原则, 有可能发生 Flag 位已经置 1, 而数据尚未完全到达数据缓冲, 从而引发数据完整性问题。

3) 双方向的数据写没有有序的关系。如图 11-1 所示, 生产者通过 PCI 桥向数据缓冲写入数据, 与消费者通过 PCI 桥更新 Status 状态位没有有序的关系。PCI 桥会为双方向的数据传递设置独立的缓冲, 两者间的数据传递没有有序的要求。

4) 读请求通过 PCI 桥时需要进行数据同步。当来自任何主设备的读请求通过 PCI 桥时, PCI 桥需要按照以下步骤处理这个读请求。

① 这个读请求总线事务首先被 PCI 桥暂存。该读请求可以穿越 PCI 桥, 或者被 PCI 桥转换为 Delayed 读请求结束主设备的请求, 并使主设备择时重试这个读请求。

② 在 PCI 桥向目标总线发起读请求之前, 需要将“与这个读请求方向相同的” Posted 写事务刷新出 PCI 桥。以图 11-1 为例, 消费者在读取 Flag 位时, PCI 桥中暂存的“从 PCI 总线 0 到 PCI 总线 1”的 Posted 写总线事务都将刷新到 PCI 总线 1 中。该操作可以保证主设备可以从目标设备获得最新的 Flag 信息, 因为“和读请求同方向”的写操作都从 PCI 桥中刷新出去。

③ PCI 桥从目标总线上获得数据, 因为在读取数据之前, 已经将 Posted 写事务刷新出 PCI 桥, 因此这个读操作可以获得最新的数据, 从而保证了数据访问的一致性。

④ PCI 桥将“与这个读完成方向相同的” Posted 写事务刷新出 PCI 桥。以图 11-1 为例, 将 Posted 写事务从桥片刷新到 PCI 总线 0, 此时主设备获得 Flag 信息后, 可以保证所有的数据已经到达数据缓冲, 因为“和读完成同方向的”写操作都从 PCI 桥中刷新出去。

⑤ 当主设备再次发出这个读请求时, PCI 桥将传送在数据缓冲中的数据。

5) Posted 写总线事务可以穿越 Non-Posted 总线事务。PCI 桥在等待其他 Non-Posted 总线事务完成时, 能够接收 Posted 写总线事务, 如果 PCI 桥不这样做, 将可能引发死锁, 如图 11-3 所示。这里有两个例外, 一个是 PCI 桥的 Posted 写数据缓冲满时, PCI 桥可以暂时不接收这个 Posted 写总线事务; 另外一个 PCI 桥正在处理一个“Locked”总线操作时, 也可以不接收 Posted 写总线事务。

11.3.2 Delayed 总线事务的传送规则

PCI 总线规定在主设备置 FRAME#信号有效后的 16 个时钟周期之内, 目标设备需要置 TRDY#有效, 否则 PCI 总线将出现夭折现象, 因此如果一个 PCI 主设备需要使用 Non-Posted 总线事务, 通过多级 PCI 桥访问最终的目的设备时, 可以使用 Delayed 总线事务。此时 PCI 桥首先 Retry 当前 Non-Posted 总线事务, 并将其转换为 Delayed 总线事务。该 Non-Posted 总线事务的发起者需要择时重试该总线事务, 而 PCI 桥将这个 Delayed 总线事务暂存。

在处理 Delayed 总线事务时, PCI 桥可以每次只处理一个 Delayed 总线事务, 当下一个 Delayed 总线事务到达时, PCI 桥可以直接拒绝此 Delayed 总线请求事务; 或者在 PCI 桥中设置一个队列, 依次将 Delayed 总线请求事务保存在这个队列中, 当这个队列满时, 再拒绝下一个 Delayed 总线事务。使用 Delayed 总线请求事务进行数据传送时, 需要遵循以下规则。

1) 只有 Non-Posted 总线事务才能使用 Delayed 总线事务。

2) 主设备访问目标设备时, 如果被 PCI 桥使用重试周期暂时中断时, 主设备必须择时重新访问这个目标设备, 因为 PCI 桥将使用 Delayed 总线请求事务继续进行数据访问, PCI 桥获得这个数据后, 主设备再从 PCI 桥中获得这个数据。

3) 如果一个 Delayed 总线请求事务被要求重试, 发起这个 Delayed 总线读请求事务的设备需要不断地择时重发这个数据访问, 直到完成这个数据访问为止。在 Delayed 总线请求事务没有到达最终的设备之前, 仅是一个数据请求, 可以随时被丢弃。而这些重试操作极大浪费了 PCI 总线的带宽, 这也是 Delayed 总线事务的缺点。

4) 当 Delayed 总线请求事务成功到达目的总线后, 这个 Delayed 总线请求事务将被转换为 Delayed 总线完成事务。此时这个 Delayed 总线完成事务除了在以下两种情况之外, 不能被随便丢弃。

① 如果主设备向一个可以预取的存储器空间进行读操作时, 产生的 Delayed 总线完成事务可以被丢弃。因为对可以预取的存储器空间进行多次读操作, 都不会产生任何副作用。

② 当主设备在 2^{15} 个时钟周期后, 依然没有进行总线重试时, PCI 桥可以丢弃这个 Delayed 总线完成事务, 而且需要通过某种机制通知主设备这个 Delayed 总线完成事务已经被丢弃, 这种情况极少发生。

5) PCI 桥在处理 Delayed 总线事务时, 必须能够接收来自这个桥同一侧的 Posted 存储器写请求。

6) Delayed 数据请求和 Delayed 数据完成之间没有序的要求。

7) Delayed 数据完成不可以超越之前的 Posted 写总线事务。

8) 如果主设备需要“Delayed 读总线事务 A”一定要在“Delayed 读总线事务 B”之前结束, 唯一的方法就是在 Delayed 读总线事务 A 完全结束后, 再启动 Delayed 读总线事务 B。因为 Delayed 读总线事务 A 有可能被设备使用重试周期结束。因此尽管 PCI 设备先发送 Delayed 读总线请求 A, 仍然有可能在后发送的 Delayed 读总线请求 B 完全结束后, 才被处理。

11.3.3 PCI 总线事务通过 PCI 桥的顺序

在 PCI 桥中, 设置了许多缓冲暂存各类总线事务, 包括 Posted 存储器写 (PMW), Delayed 读请求 (DRR), Delayed 读完成 (DRC), Delayed 写请求^① (DWR) 和 Delayed 写完成 (DWC)。这些不同种类的总线事务在同一方向穿越 PCI 桥时, 无论是从上游总线到下游总线还是从下游总线到上游总线穿越 PCI 桥时, 都必须遵循一定的顺序, 以满足生产者/消费者模型在 PCI 总线上的正确运行。有关 PCI 桥的序如表 11-4 所示。在该表中出现的“**Yes**”、“**No**”和“**Yes/No**”的定义如下所示。

- “**Yes/No**”表示 Row 和 Column 之间的总线事务没有序的关系, 如 DWC (Row E) 总线事务和 PMW (Column 2) 总线事务在通过 PCI 桥时没有先后顺序。
- “**Yes**”表示 Row 中的总线事务先于 Column 中的总线事务通过 PCI 桥。比如 PMW 总线事务可以超越 DRR 和 DWR 总线事务。
- “**No**”表示 Row 中的总线事务后于 Column 中的总线事务通过 PCI 桥。如 PMW 总线事务不能超越之前的 PMW 总线事务。

^① I/O 和配置写总线事务使用 DWR。

- 表中的“**Yes**”和“**No**”的上标对应一个序的规则，例如 No¹中的上标 1 对应规则 1。这些规则将在下文陆续介绍。

表 11-4 PCI 桥使用的数据访问顺序

Row pass Col?	PMW Column ²	DRR Column ³	DWR Column ⁴	DRC Column ⁵	DWC Column ⁶
PMW (Row A)	No ¹	Yes ⁵	Yes ⁵	Yes ⁷	Yes ⁷
DRR (Row B)	No ²	Yes/No			
DWR (Row C)	No ³				
DRC (Row D)	No ⁴	Yes ⁶		Yes/No	
DWC (Row E)	Yes/No ⁴				

1. Posted 存储器写通过 PCI 桥时需要按序完成

Posted 存储器写通过 PCI 桥时需要遵循“先进先出”的原则，否则将会引发数据完整性问题。这个要求对满足生产/消费者模型的正常运行至关重要。

对于生产者，置 Flag 位为 1 和将数据写入数据缓冲都使用 Posted 存储器写总线事务。如果 Flag 位和数据缓冲在同一条 PCI 总线上时，Posted 存储器写不按序到达，可能导致 Flag 位被生产者置 1，而数据缓冲并没有收到数据，从而在消费者使用数据缓冲中的数据时，可能会得到无效数据。

2. DRR 不能超越 PMW

PCI 桥首先将 DRR 保存在缓冲区中，之后 PCI 桥将缓存在该桥中的所有 PMW 都刷新出去后，才能执行这个 DRR，即“先写后读”，否则读入的数据有可能不是最新的。系统软件可以使用这一功能实现“读刷新”操作。PCIe 总线还支持读“0 字节”操作，其主要目的就是完成这种“读刷新”操作。

3. DWR 不能超越 PMW

DWR 首先在 PCI 桥中锁存，之后 PCI 桥将缓存的所有 PMW 发送出去后，才能执行这个 DWR。在生产/消费者模型中，生产者除了可以使用 PMW 总线事务设置 Flag 位之外，也可以使用 DWR 总线事务设置 Flag 状态位（Flag 状态位可能存放在 I/O 地址空间中，而且假设在图 11-1 中，Flag 位和数据缓冲都在 PCI 总线 0 上）。

此时 DWR 不能超越 PMW，因为生产者必须将所有数据都写入数据缓冲后，才能设置 Flag 状态位。如果 DWR 可以超越 PMW，则会出现生产者没有将数据完全写入到数据缓冲中，而 Flag 位已经置 1，从而可能引发数据完整性问题。规则 2、3 与规则 5 直接相关，DRR、DWR 和不能超越 PMW，也意味着 PMW 需要超越 DRR 和 DWR。

4. DRC 不能超越 PMW

PCI 总线使用 DRR 总线事务处理存储器和 I/O 读请求，而该总线事务在获得所读取的数据和完成状态后，将被转化为 DRC 总线事务，并在主设备重新发起读操作时，将数据传递给主设备。DRC 要求数据在穿越 PCI 桥传递给主设备之前，PCI 桥将缓存的 PMW 总线事务刷新出去。

以图 11-1 为例，如果消费者使用 DRC 总线事务获得 Flag 位，当 Flag 位为 1 时，必须要求生产者将数据全部写入数据缓冲中。如果 DRC 可以超越 PMW，则可能出现消费者通过

DRC 获得 Flag 位, 而且 Flag 位为 1 时, 生产者提供的数据仍在 PCI 桥中的情况。因为这些数据需要通过 PCI 桥才能达到缓冲, 而 Flag 位与生产者在同一条总线上, 可以立即生效。

当消费者发现 Flag 位为 1 时, 会立即读取数据缓冲, 在图 11-1 中, 如果这个 DRC 请求能够超越 PMW, 那么 PCI 桥首先将 Flag 位已经为 1 的消息传递给消费者, 之后消费者开始从缓冲中读取数据, 而这时生产者使用 PMW 方式写入缓冲的数据还可能在 PCI 桥中, 从而造成数据完整性的问题。

在 PCI 总线中, 解决该问题的方法是 DRC 不能超越 PMW, 这样消费者在收到 Flag 位为 1 的信息之前, 在 PCI 桥中的 PMW 一定被 DRC 刷新到 PCI 总线 0 中, 从而不会引发数据完整性问题。规则 4 与规则 7 之间相关, DRC 不能超越 PMW, 也意味着 PMW 需要超越 DRC。

在 PCI 总线中, DWC 与 PMW 之间没有序的要求。因为 DWC 中并不包含有效数据, 仅是通知发送 DWR 的设备, 该写请求已经结束。DWC 不在生产/消费者模型中出现。

5. PMW 可以超越 DRR 和 DWR

PCI 总线为了避免死锁, 可以使刚进入 PCI 桥的 PMW 总线事务, 超越已经在 PCI 桥中暂存的 DRR 和 DWR 总线事务, 即 PMW 总线事务可以提前执行。如果 PMW 不能超越 DRR 和 DWR, 则会产生死锁。实际上因为规则 2 和 3 的原因, 规则 5 的引入是顺理成章的。

值得注意的是该规则的引入还解决了在 PCI 总线中使用的不同版本的 PCI 桥的问题, 本小节对这种情况不做深入讨论。

6. Delayed 读写完成可以超越 Delayed 读写请求

如果 DWC 和 DRC 总线事务到达 PCI 主设备时, 这个主设备正在等待 DWR 和 DRR 总线事务完成, DWC 和 DRC 必须可以优先到达 PCI 主设备, 否则将引发死锁, 下文以图 11-4 所示的实例说明这种死锁。

我们考虑 PCI 主设备 X 和 Y 通过 PCI 桥 A 和 B 进行数据读写, 其步骤如下。

1) PCI 主设备 X 和 Y 同时发起一个 Non-Posted 读写请求。

2) PCI 桥 A 和 B 将这个 Non-Posted 总线读写请求转换为 Delayed 读写请求, 并暂存在桥内缓冲中, 分别为 DRR-X 和 DRR-Y, 同时使用重试周期结束 PCI 主设备 X 和 Y 的读写请求。之后 PCI 主设备 X 和 Y 将定时重发这个 Non-Posted 总线读写请求, 直到完成本次总线读写请求。

3) PCI 桥 A 和 B 依次获得 PCI 总线 1 的使用权, 并将 DRR-X 和 DRR-Y 请求发向对方。PCI 桥 B 和 A 将来自对方的 Delayed 读写请求锁存在桥内缓冲中, 并发起重试周期结束来自 PCI 桥 A 和 B 的 Delayed 读写请求。

4) PCI 桥 A 和 B 将定时重发这个 Delayed 总线读写请求, 直到获得 Delayed 总线读写完成信息。

5) PCI 桥 A 和 B 将分别获得 PCI 总线 0 和 2 的使用权, 将 DRR-Y 和 DRR-X 请求发送到最终 PCI 设备。假设 PCI 桥 A 获得 PCI 总线 0 的使用权, 并完成了 PCI 主设备 Y 发起的 DRR-Y 请求, 此时 PCI 桥 A 将从 PCI 主设备 X 中得到 Delayed 读写完成信息, 并将 DRR-Y 请求转换为 DRC-Y 请求, 并将其锁存在 PCI 桥 A 的缓存中。等待 PCI 桥 B 对 PCI 桥 A 进行重试。

6) 如果此时 PCI 桥 A 在没有完成 DRR-X 请求 (该请求是发向 PCI 主设备 Y) 时, 不能接收 DRC-Y 请求将引发死锁。因为 DRC-X 请求也会因为相同的原因不会被 PCI B 桥接收, 从而 PCI 桥 A 无法完成 DRR-X 请求。

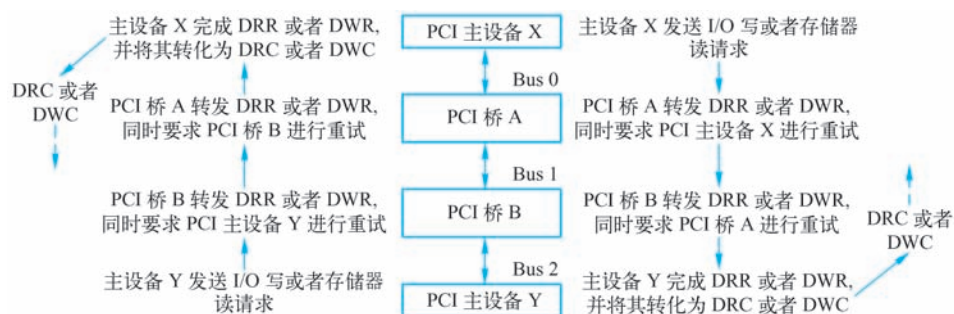


图 11-4 DWC/DRC 不能超越 DRR/DWR 而造成死锁

如果 Delayed 总线完成可以超越 Delayed 请求时，将有效地避免这类死锁。因为 Delayed 总线完成是发生在 Delayed 总线请求之后的事件，这两个请求有因果关系，希望读者认真体会规则 6。

7. PMW 总线事务可以超越 DRC/DWC 总线事务

PCI 总线规定 PMW 总线必须可以超越之前已经在 PCI 桥中暂存的 DRC/DWC 总线事务，以避免死锁。引入该规则的主要原因是 PCI 总线层次结构中可能存在不同版本的 PCI 桥，它们的缓冲管理策略并不一致，从而有可能造成死锁。本节对这种情况不做进一步描述。

11.3.4 LOCK, Delayed 和 Posted 总线事务间的关系

PCI 桥可以将一个 LOCK 总线事务从上游传递到下游，但是不能将一个 LOCK 总线事务从下游传递到上游。PCI 桥将下游传递到上游的 LOCK 总线事务转换为普通的总线事务，即去掉这个 LOCK 标志。

当 PCI 桥从一个主设备接收一个 LOCK 总线事务之后，将这个总线事务发送到下游总线的目标设备。如果该 LOCK 总线事务是 Non-Posted 总线事务，此时 PCI 桥并不能立即完成这个 LOCK 总线事务，因为 PCI 桥还需要将“Non-Posted 总线请求”对应的总线完成信息传递给发起者之后，LOCK 总线事务才能最终完成。

在 PCI 桥没有将 LOCK 总线完成传递给发起者之前这段时间里，PCI 桥仅接收这个发起 LOCK 总线事务的主设备的总线请求，而重试其他主设备发出的总线事务。该 PCI 桥不会使用 Delayed 总线事务接收其他 Non-Posted 总线事务，也不会暂存这些总线事务。

一个 LOCK 总线事务从 PCI 桥的上游到达 PCI 桥，并在 PCI 桥将这个 LOCK 总线事务传递到下游总线这段时间里，PCI 桥需要进行以下处理。

1) 将所有同方向的 PMW 总线事务刷新到下游总线。

2) 对于 Delayed 总线事务，PCI 桥需要进行特别处理。丢弃所有暂存在 PCI 桥中的 Delayed 总线事务；允许 LOCK 总线事务超越任何读写请求。或者完成所有 Delayed 读写请求，再将 LOCK 总线事务发送给下游总线。

在 PCI 桥的下游总线接收 LOCK 总线请求之前，PCI 桥仍然可以暂存来自下游总线的数据请求；而在下游总线接收 LOCK 总线请求之后，PCI 桥不能接收任何来自下游总线的数据请求，直到发起 LOCK 总线请求的主设备解锁。

在一个最简单的 PCI 桥的实现中，一个 LOCK 总线事务在 PCI 桥的下游总线建立完毕后，PCI 桥不能接收上/下游总线的数据请求，除了来自发起 LOCK 总线事务的 PCI 主设备

的数据请求。而且 PCI 桥必须完成发向上游总线的 PMW, DRC 和 DWC 总线请求。采用这个规则可以保证使用 LOCK 总线事务时不会引发死锁。但是使用这些规则将极大影响 PCI 总线的传送性能, 为此在处理器系统的设计中, 最好不使用 LOCK 总线事务。

11.4 PCIe 总线的序

PCIe 总线的序基于 PCI 总线的序, 并进行了许多扩展。在 PCI 总线上, 仅能使用强序传送规则, 而 PCIe 总线支持 Relaxed ordering 方式进行数据传递, 使用这种方法时, 不同的 TLP 在通过 RC 和 Switch 到达 EP 时, 不一定遵循 PCI 总线的强序原则, 这也意味着先发出去的 TLP 并不一定能够最先到达目的地。PCIe 总线使用 Relaxed ordering 数据传送方式, 在一定程度上可以提高数据传送效率。

在 TLP 的 Attr 字段中有一个 Relaxed Ordering 位, 表示该 TLP 是否支持 PCIe 总线的 Relaxed Ordering 方式, 但是 TLP 是否可以使用 Relaxed Ordering 还与这个 TLP 经过的设备有关。如果一个 TLP 经过的 Switch 不支持 PCIe 的 Relaxed Ordering 数据传送方式, 通过这个 Switch 的 TLP 报文依然需要使用强序方式通过这个 Switch。

系统软件可以通过使能 Device Control 寄存器中的 Enable Relaxed Ordering 位, 来禁止或者使能 TLP 报文的 Relaxed Ordering 功能, Device Control 寄存器在 PCIe 设备的 PCI Express Capability 结构中。目前大多数 PCIe 设备不支持 Relaxed Ordering 方式进行 TLP 的传递。

PCIe 总线的 Relaxed Ordering 数据传送方式是有条件的, PCIe 总线的每一个 TLP 报文都有一个唯一的 TC, 而这个 TC 又和一个唯一的 VC 对应^①。Relaxed Ordering 与报文使用的 VC 相关。VC 相同的 TLP 间的传送遵循 Relaxed Ordering 的原则, 而 VC 不同的 TLP 间没有序的要求。在 PCIe 总线中, 所有数据传送类型, 如存储器、I/O、配置和 Message 总线事务都需要遵循规定的传送顺序。

11.4.1 TLP 传送的序

VC 不同的 TLP 间没有序的要求, 在 PCIe 总线中, “序”是指 VC 相同的 TLP 之间的传送顺序, 其关系如表 11-5 所示。

表 11-5 PCIe 总线的序

Row Pass Col?	Posted Request Col 2	Read Request Col 3	NPR with Data Col 4	Completion Col 5
Posted Request Row A	a. No b. Y/N	Yes	Yes	a. Y/N b. Yes
Read Request Row B		Y/N		
NPR with Data Row C				
Completion RowD		Yes		a. Y/N b. No

① 不同的 TC 可能会共用一个 VC, 而一个 TC 只能使用一个 VC。TC 和 VC 的对应关系是多对一的关系。如 TC1 和 TC2 可以都可以使用 VC1, 而 TC1 不能在使用 VC0 的同时还使用 VC1。

各个表项的含义如下。

- Posted Request 由存储器写请求 TLP 或者 Message 使用。
- Read Request 由 I/O、配置和存储器读请求使用。
- NPR (Non-Posted Request) with Data 由 I/O、配置写和原子操作使用。
- a. 表示 TLP 的 RO 位为 0，即不使能 Relaxed Ordering 的情况。
- b. 表示 TLP 的 RO 位为 1 或者 IDO 位为 1，即使能 Relaxed Ordering 或者使能 ID-Based Ordering 的情况。不同的规则使用 a, b 子规则略有差异。
- Yes 表示 Row 中的 TLP 必须能够穿越 Col 中的 TLP。
- Y/N 表示 Row 中的 TLP 和 Col 中的 TLP 没有序的关系。
- No 表示 Row 中的 TLP 一定不能穿越 Col 中的 TLP。

下文出现的 XY a/b 中，X 与行对应，其值为 A~D，Y 与列对应，其值为 2~5。如 A2 a 表示“Posted Request”在 RO 位为 0 的情况下是否能够超越“Posted Request”。

通过表 11-5 与表 11-4 的比较，可以发现在 RO 位为 0 时（即不使用 Relaxed Ordering），PCIe 总线的序与 PCI 的序基本兼容。但是因为在一个 TLP 中有时 RO 位和 IDO 位不为 0，因此 PCIe 总线的序需要根据 a 和 b 两种情况分别进行讨论。

1. A2

A2 需要分为两种情况讨论，其中 a 对应 TLP 的 RO 和 IDO 位都为 0 情况，而 b 对应 TLP 的 RO 或者 IDO 位为 1 的情况。

A2 a 的值为 No，表示 Posted Request 报文不能超越之前的 Posted Request 报文，这与 PCI 总线中 PMW 不能超越之前的 PMW 要求相同。PCI 总线的 PMW 与 PCIe 的 Posted Request 报文基本一致，存储器写和 Message 使用这类报文。

A2 b 的值为 Y/N，该规则需要分为两种情况进行讨论，RO 位为 1 或者 IDO 位为 1。当 RO 位为 1 时，该 Posted Request 报文可以超越之前的 Posted Request 报文。在设计中应用该规则是十分危险的，该规则也意味着“写”可以超越“写”。

如在第 11.1.1 节描述的生产/消费者模型中，生产者首先将数据写入数据缓冲，然后将 Flag 位置 1。如果“将 Flag 位置 1”的写操作可以超越“写入数据缓冲”的写操作，那么消费者可能会从无效的数据缓冲中读取数据，从而出现错误。

在 Switch 和支持 Peer-to-Peer 传送的 RC 中，设置了一个寄存器位“**No RO-enabled PR-PR Passing**”[⊖]，当该位为 1 时，当 TLP 通过这些 Switch 和 RC 时，Posted Request 报文不能超越之前的 Posted Request 报文，即便这些 TLP 的 RO 位为 1。

当 IDO 位为 1 时，该 Posted Request 报文可以超越之前的 Posted Request 报文。使用该规则的前提是，这两个 Posted Request 报文使用的 Requester ID 号不同，即这两个 Posted Request 报文是由不同的 PCIe 设备发出的，有关 IDO 序的详细说明见第 11.4.2 节。

2. A3 和 A4

A3 和 A4 的值为 Yes，表示 Posted Request 报文可以超越之前的 Non-Posted 读和写请求。该规则与 PCI 总线的 PMW 可以超越 DRR 和 DWR 兼容，其主要目的是避免死锁。详见 PCI

⊖ 该位在 Device Capabilities 2 寄存器中。

总线中的规则 5。

3. A5

A5 需要分为两种情况讨论，a 适用于 PCIe 总线中的 RC 和 Switch；而 b 适用于 PCIe 桥。

A5 a 的值为 Y/N。在 PCIe 总线中，Posted Request 报文可以超越之前的完成报文也可以不超越，在这两种情况下，都不会造成死锁。该规则与 PCI 总线中 PMW 必须超越 DRC 和 DWC 不同（PCI 总线中的规则 7），因为在 PCI 体系结构中会出现不同版本的 PCI 桥，而在 PCIe 体系结构中不会出现这种情况。

A5 b 的值为 Yes。表示在 PCIe 桥中，PCIe 总线向 PCI 总线的方向传递报文时，Posted Request 报文必须可以超越完成报文，以避免死锁。PCIe 桥内部由多个虚拟 PCI 桥组成，参见图 4-13，因此 PCIe 总线中的 A5 b 与 PCI 总线中的规则 7 兼容。

4. B2, C2

B2 需要分为两种情况讨论，其中 a 对应 TLP 的 IDO 位为 0 情况，而 b 对应 TLP 的 IDO 位为 1 的情况。

B2 a 的值为 No，表示 Read Request 报文不能超越之前的 Posted Request 报文，这与 PCI 总线中 DRR 不能超越之前的 PMW 要求相同。PCI 总线的 DRR 与 PCIe 总线的 Read Request 报文基本一致，存储器、I/O 和配置读使用这类报文。

B2 b 对应 TLP 的 IDO 位为 1 的情况。当 IDO 位为 1 时，该 Read Request 报文可以超越之前的 Posted Request 报文，否则不能超越。使用该规则的前提是，Read Request 报文和 Posted Request 报文使用的 Requester ID 号不同。

C2 也需要分为两种情况讨论，其中 a 对应 TLP 的 IDO 位为 0 情况，而 b 对应 TLP 的 IDO 位为 1 的情况。其实现机制与 B2 类似，本节对此不做进一步说明。

5. B3, B4, C3, C4

在 PCIe 总线中，Non-Posted Request 报文可以超越之前的 Non-Posted Request 报文，也可以不进行这种超越。该规则从 PCI 总线中继承而来，而在 PCI 总线中 DRR/DWR 可以超越之前的 DRR/DWR。PCIe 设备在实现中，需要与该规则兼容。

但是在 PCIe 总线中，存储器读操作使用 Split 方式进行传送，因此该规则的引入为 PCIe 设备的设计带来了不小的麻烦。当一个 EP 进行 DMA 读操作时，需要首先向 RC 发送存储器读请求，如 R1~R4，而 RC 收到这些读请求时，将回送读完成 TLP，如 C1~C4。为简便起见，我们认为每一个存储器读请求的大小为 64B，而读完成的大小也为 64B，而且不考虑对界的问题，这样 EP 发送的存储器读请求将与 RC 的读完成一一对应，我们假设 R1 对应 C1，R2 对应 C2，并以此类推 R4 对应 C4。

EP 首先按照 R1~R4 的顺序发送这些存储器读请求。但是 R1~R4 在通过 Switch 和 RC 之后可能出现乱序，如果 Non-Posted Request 报文可以超越之前的 Non-Posted Request 报文，RC 最终收到的存储器读请求可能是乱序的，如 R2, R4, R3 和 R1，因此 RC 发送给 EP 的读完成报文可能为 C2, C4, C3 和 C1，这个顺序与 EP 发向 RC 的存储器读请求的顺序并不相同。因此 EP 必须处理这种乱序，这为 EP 的设计带来了不小的困难。

6. B5, C5

在 PCIe 总线中，Non-Posted Request 报文与之前的完成报文没有序的要求。该规则从 PCI 总线中继承而来，在 PCI 总线中 DRR/DWR 可以超越之前的 DRC/DWC，也可以不超

越。这些报文的传递不会影响生产/消费者模型的正常运行。

7. D2

D2 需要分为两种情况进行讨论，分别是 D2 a 和 D2 b。

其中 D2 a 为 No，表示在 PCIe 总线中，CplD 报文不能超越 Posted Request 报文，该规则与 PCI 总线中的规则 4 兼容。这也是保证生产/消费者模型正常运行的必要条件。

而 D2 b 为 Y/N。如果 TLP 的 RO 位为 1 时，该 CplD 报文可以超越 Posted Request 报文。设计者需要慎重使用该规则，因为该规则的应用有可能破坏生产/消费者模型的正常运转。只有传递与生产/消费者模型无关的报文时，才能应用该规则。

此外如果 TLP 的 IDO 位为 1 时，该 CplD 报文可以超越之前的 Posted Request 报文，否则不能超越。使用该规则的前提是，CplD 报文和 Posted Request 报文使用的 Requester ID 号不同。值得注意的是，Cpl 报文由 I/O 或者配置写完成报文使用，该报文中不含有数据，仅包含完成信息。该报文的使用方法与 PCI 总线的 DWC 类似。该报文与 Posted Request 报文没有有序的要求，该规则与 PCI 总线的规则 4 兼容。

8. D3, D4

在 PCIe 总线中，完成报文可以超越之前的 Non-Posted Request 报文。该规则从 PCI 总线中继承而来，与规则 6 对应。该规则的引入主要为了防止死锁。

9. D5

如果完成报文与之前的完成报文的 Transaction ID 不同时，该报文可以超越之前的完成报文；如果相同，不能进行这样的超越。

当一个 PCIe 设备向目标设备发送存储器读请求时，目标设备可能会使用一个或者多个存储器读完成报文将数据回送。如果使用多个存储器读完成报文时，这些存储器完成报文按“地址升序”顺序先后到达源设备。

如果设备 A 需要从设备 B 读取 256B[⊖]的数据，其访问的地址为 0x1000-0000~0x1000-00FF 时，设备 A 可以向设备 B 发送一个存储器读请求 TLP，而设备 B 将以 64B 为单位向设备 A 发送存储器读完成 TLP，这些完成报文必须以 C1~C4 的顺序到达设备 A，C1~C4 存储器读完成 TLP 对应的数据区域如下。

- C1 与 0x1000-0000~0x1000-003F 对应。
- C2 与 0x1000-0040~0x1000-007F 对应。
- C3 与 0x1000-0080~0x1000-00BF 对应。
- C4 与 0x1000-00C0~0x1000-00FF 对应。

如果设备 A 需要从设备 B 读取 512B 的数据，访问的地址为 0x1000-0000~0x1000-01FF 时，这段数据区域大于设备 A 的 Max_Read_Request_Size 参数，因此设备 A 需要向设备 B 发出两个存储器读请求，这两个存储器读请求使用两个不同的 Tag 字段进行区分，分别为 R_{T0} 和 R_{T1}。假设 R_{T0} 的 Tag 字段为 0，其请求的数据区域为 0x1000-0000~0x1000-00FF；而 R_{T1} 的 Tag 字段为 1，其请求的数据区域为 0x1000-0100~0x1000-01FF。

假设设备 A 首先发送 R_{T0}，然后再发送 R_{T1}。但是设备 B 仍然可能首先收到 R_{T1}，然后再

⊖ 假设设备 A 的 Max_Read_Request_Size 参数为 256B，而设备 B 可以为 RC 也可以为普通的 EP。

收到 R_{T_0} ，因为 PCIe 总线允许存储器读请求超越存储器读请求。设备 B 收到这些存储器读请求后，向设备 A 发送存储器读完成 TLP（以 64B 为单位），分别为 $C1_{T_0} \sim C4_{T_0}$ 和 $C1_{T_1} \sim C4_{T_1}$ 。

其中 $C1_{T_0} \sim C4_{T_0}$ 使用的 Tag 字段为 0，而 $C1_{T_1} \sim C4_{T_1}$ 使用的 Tag 字段为 1，分别与 R_{T_0} 和 R_{T_1} 对应，这也意味着这两组存储器读完成使用的 Transaction ID 不同，因此可以彼此超越，这两组存储器读完成 TLP 对应的数据区域如下。

- $C1_{T_0}$ 与 $0x1000-0000 \sim 0x1000-003F$ 对应； $C2_{T_0}$ 与 $0x1000-0040 \sim 0x1000-007F$ 对应； $C3_{T_0}$ 与 $0x1000-0080 \sim 0x1000-00BF$ 对应； $C4_{T_0}$ 与 $0x1000-00C0 \sim 0x1000-00FF$ 数据区域对应。
- $C1_{T_1}$ 与 $0x1000-0100 \sim 0x1000-013F$ 对应； $C2_{T_1}$ 与 $0x1000-0140 \sim 0x1000-017F$ 对应； $C3_{T_1}$ 与 $0x1000-0180 \sim 0x1000-01BF$ 对应； $C4_{T_1}$ 与 $0x1000-01C0 \sim 0x1000-01FF$ 数据区域对应。

此时设备 A 收到的存储器读完成，有多种可能，如表 11-6 所示。

表 11-6 设备 A 收到的存储器读完成序列

序列 1	序列 2	序列 3	序列 4	序列 5	序列 6	序列 7	序列 8
$C1_{T_0}$	$C1_{T_0}$	$C1_{T_1}$	$C1_{T_1}$	$C1_{T_0}$	$C1_{T_0}$	$C1_{T_1}$	$C1_{T_1}$
$C2_{T_0}$	$C1_{T_1}$	$C2_{T_1}$	$C2_{T_1}$	$C1_{T_1}$	$C2_{T_0}$	$C2_{T_1}$	$C1_{T_0}$
$C3_{T_0}$	$C2_{T_0}$	$C3_{T_1}$	$C1_{T_0}$	$C2_{T_1}$	$C3_{T_0}$	$C3_{T_1}$	$C2_{T_0}$
$C4_{T_0}$	$C2_{T_1}$	$C4_{T_1}$	$C2_{T_0}$	$C2_{T_0}$	$C1_{T_1}$	$C1_{T_0}$	$C2_{T_1}$
$C1_{T_1}$	$C3_{T_0}$	$C1_{T_0}$	$C3_{T_1}$	$C3_{T_0}$	$C2_{T_1}$	$C2_{T_0}$	$C3_{T_0}$
$C2_{T_1}$	$C3_{T_1}$	$C2_{T_0}$	$C4_{T_1}$	$C4_{T_0}$	$C3_{T_1}$	$C3_{T_0}$	$C4_{T_0}$
$C3_{T_1}$	$C4_{T_0}$	$C3_{T_0}$	$C3_{T_0}$	$C3_{T_1}$	$C4_{T_1}$	$C4_{T_0}$	$C3_{T_1}$
$C4_{T_1}$	$C4_{T_1}$	$C4_{T_0}$	$C4_{T_0}$	$C4_{T_1}$	$C4_{T_0}$	$C4_{T_1}$	$C4_{T_1}$

上表仅列出了设备 A 可能从设备 B 中收到的存储器读完成序列。由此可见对于 Tag 字段不同的存储器完成报文，在到达设备 A 时顺序并不确定。但是对于 Tag 字段相同的存储器读完成 TLP，这些存储器完成报文是严格按照地址“升序”的顺序到达设备 A。PCIe 总线的这种乱序为 PCIe 设备的设计带来了不小的麻烦，设计者必须认真地处理这些乱序可能。

11.4.2 ID-Base Ordering

IDO 模型由 PCIe V2.1 版本引入。该模型引入了“数据流”的概念，即相同的数据源发出的 TLP 属于相同的“数据流”，而不同数据源发出的 TLP 属于不同的“数据流”。PCIe 链路可以根据“数据流”对 TLP 进行区分。IDO 模型允许分属不同“数据流”的 TLP 之间没有序的要求，即可以“自由乱序”。

IDO 模型的引入有利于 Switch 对发向不同 Egress 端口的报文进行优化处理。我们假设 Switch 的一个 Ingress 端口收到了若干个数据报文，这些报文分别发向不同的 Egress 端口，如图 11-5 所示。

其中 TLP1~3 分别发向 Egress 端口 1~3，在不使用 IDO 模型的情况下，Ingress 端口需要等待之前的报文被发送出去之后，才能发送之后的报文。TLP3、TLP2 和 TLP1 依次进入 Ingress 端口，如果不使用 IDO 模式时，TLP2 需要等待 TLP3 完全离开 Ingress 端口后，才能

被发送；同理 TLP1 需要等待 TLP2 离开 Ingress 端口才能被发送。显然这种数据传送方式并不合理。



图 11-5 IDO 模型的优点

这种拥塞也被称为 HOL（Head-of-Line）Blocking。引起这种现象的主要原因是 Ingress 端口每次只能处理一个报文引起的。如果在 Ingress 端口只有一个发送部件与 Egress 端口 1~3 对应时，即便使用 IDO 模型并不会提高效率，因为这些报文依然需要通过这一个发送部件串行发送。由此可见对于这种类型的 Switch，使用 IDO 模型并不会提高效率。

但是如果 Ingress 端口中有 3 个发送部件，分别与 Egress 端口 1~3 对应时，实际上是 Ingress 端口为每一个 Egress 端口提供分离的发送缓冲，这个缓冲也被称为 VOQ（Virtual Output Queue）。此时 TLP1~3 的发送可以同时进行，而 Ingress 端口使用 IDO 模型，可以不考虑传送“序”^①而同时发送 TLP1~3，从而极大地提高了 Switch 的转发效率。目前多数 Switch 的 Ingress 端口都支持 VOQ 技术。

在 PCIe V2.1 总线规范提出之前，PLX^②公司已经使用类似 IDO 模型的技术以优化 Switch 的数据传送，即 PLX-Specific Relaxed Ordering 技术。下文以 PEX8518 芯片为例说明该技术的具体实现，该芯片是 PLX 公司设计的 PCIe Switch。在 PEX8518 中，每一个 Ingress 端口都为不同的 TC 设置了一个“PLX-Specific Relaxed Ordering”使能位，当该位为 1 时，当一个 Ingress 端口收到的 TLPs 发向的 Egress 端口不同，则没有序的要求；而 Egress 端口相同的 TLPs 必须按序进行。PLX 使用的这种技术与 IDO 模型类似。

11.4.3 MSI 报文的序

在 PCIe 总线中，还有一种序引发的数据完整性问题需要特别注意，即由 MSI 报文引发的数据完整性问题。PCIe 设备使用 MSI 机制时，通过向中断控制器发送 MSI 报文以提交中断请求。然而对于 PCIe 体系结构而言，这个 MSI 报文与普通的存储器写报文并没有本质的区别，这个报文也可以使用不同的 TC。如果设备的数据传送使用 TC0，而 MSI 报文使用 TC1 时，将可能引发数据完整性的问题。

假设一个 PCIe 设备正在使用 DMA 写操作，将一组数据传递到主存储器，此时该设备将使用存储器写 TLP 进行数据传送，当数据传送完成后，使用 MSI 报文通知处理器 DMA 写操

① TLP1~3 使用的 Requester ID 不同，因此在 IDO 模型中，没有序的要求。

② PLX 是 PCIe Switch 芯片的主要提供商。

作已经结束。

如果进行数据传递的 TLP 使用 TC0，而 MSI 报文使用 TC1 时，这两种 TC 可能使用的 VC 并不相同，而不同 VC 间的数据传递并没有序的要求。因此该 PCIe 设备虽然从设计逻辑上保证，将传递数据的存储器写 TLP 发送完毕后再发送 MSI 报文，但是 RC 仍然会首先收到 MSI 报文，然后再收到传递数据的存储器写 TLP。

此时如果处理器在收到 MSI 报文后，立即在中断处理服务例程中使用该 PCIe 设备写入的数据，将可能引发数据完整性问题。

PCIe 总线规范并没有约定如何处理传递 MSI 报文而产生的数据完整性问题。在上述实例中，如果 MSI 报文使用的 TC 与数据传递使用的 TC 相同，将不会出现这个数据完整性问题。如果在设计中 MSI 报文使用的 TC 与数据传递使用的 TC 不一致，需要注意该问题。

一个可行的方法是在数据传递结束后，使用“zero-length 存储器读请求 TLP”对目标设备进行读操作，这个读操作可以将数据写入最终目的地。当该读操作结束后，即 PCIe 设备收到存储器读完成 TLP 后，再发送 MSI 报文。

如一个 PCIe 设备完成 DMA 写操作之后，再向目标地址（某个存储器地址）发送一个“zero-length 存储器读请求”报文，该报文可以保证之前的存储器写报文都被刷新到主存储器后，才能从主存储器获得应答信息，因为存储器读请求 TLP 不能超越存储器写报文。当 PCIe 设备收到与这个存储器读请求对应的存储器读完成 TLP 后，再发送 MSI 报文进行中断请求。

使用上述方法虽然可以避免因为传送 MSI 报文带来的数据完整性问题，但是将带来较大的中断延时。因为在 PCIe 体系结构中，一个设备从“发送存储器读请求 TLP”到“获得存储器读完成 TLP”的延时较长。而且使用这种方法也增加了硬件逻辑设计的难度。

目前支持多个 VC 的 PCIe 设备，通常将 MSI 报文和数据传送报文使用的 TC 设置为相同的值，以避免数据完整性问题。如在 Intel 的高精度声卡控制器中，数据传送使用的报文和 MSI 报文都只能使用 TC0。

11.5 小结

本章重点介绍 PCI/PCIe 总线的序。在局部总线中，数据传送顺序以及与其相关的数据完整性一直是系统程序员的学习重点与难点。对于系统程序员而言，这部分内容必须熟练掌握。有许多系统 Bug 仍然因为系统程序员的疏忽，或者并没有深入理解数据完整性的原理，而无意产生，并很难复现。这些 Bug 将对一个处理器系统的稳定运行产生致命影响。

在 PCI/PCIe 总线中规定了数据传送序，使用生产/消费者模型进行数据传递，合理解决了数据完整性问题。而设计者在实现 PCI/PCIe 体系结构时，必须遵循这些传送序，并且符合生产/消费者模型要求的数据传送方式。